Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

ОТЧЁТ

по проектной работе по дисциплине

«Математический анализ»

«Математический анализ в методах машинного обучения»

Выполнили: Вейгандт В.
Зайцев Н.
Горобец О.
Козлов В.
Шагеев Э.
Группа: Р 1.2

Преподаватель: Карпычев А.В.

Санкт-Петербург
2026

Содержание

Введение **3**

1 Постановка задачи машинного обучения **4**
1.1 Объекты, признаки и ответы 4
1.2 Модель как семейство функций 4
1.3 Функция потерь 5
1.4 Обучающая и тестовая выборки 5

2 Линейные модели **5**
2.1 Линейная регрессия 5
2.2 Среднеквадратичная ошибка 6
2.3 Градиент среднеквадратичной ошибки 7
2.4 Логистическая регрессия 7
2.5 Максимальное правдоподобие и cross-entropy 8

3 Градиентные методы оптимизации **9**
3.1 Градиентный спуск 9
3.2 Влияние размера шага 10
3.3 Momentum 10
3.4 Nesterov Momentum 11
3.5 AdaGrad 11
3.6 RMSProp 11
3.7 Adam 12
3.8 Сравнение методов 12

4 Решающие деревья и градиентный бустинг **13**
4.1 Решающие деревья 13
4.2 Критерий разбиения 13
4.3 Жадное построение дерева 14
4.4 Почему обычный Gradient Descent здесь неприменим 14
4.5 Случайность и разнообразие деревьев 14
4.6 Идея ансамбля 15
4.7 Gradient Boosting 15

5 Дополнительные методы **16**
5.1 Principal Component Analysis 16
5.2 Метод Монте-Карло 16
5.3 Муравьиный алгоритм 17

6 Практическая часть **17**
6.1 Используемые средства 17
6.2 Эксперимент 1: поверхность loss линейной регрессии 18
6.3 Эксперимент 2: влияние learning rate 18
6.4 Эксперимент 3: сравнение оптимизаторов 19
6.5 Эксперимент 4: построение небольшого дерева 20

6.6 Эксперимент 5: пошаговый Gradient Boosting 20
6.7 Дополнительные демонстрации 21

7 Распределение работы между участниками **21**

Заключение **22**

Список источников **22**

А Программная реализация **23**

Введение

Машинное обучение рассматривает методы построения алгоритмов, поведение которых определяется не только явно заданными правилами, но и данными. В задачах обучения с учителем имеется набор объектов, для которых известны правильные ответы, и требуется построить модель, способную делать предсказания для новых объектов.

С математической точки зрения значительная часть таких задач естественно сводится к выбору функции из некоторого параметризованного семейства и поиску значений параметров, при которых ошибка модели минимальна. Поэтому при изучении машинного обучения возникают конструкции, непосредственно связанные с математическим анализом: функции нескольких переменных, частные производные, градиент, экстремумы, линии уровня и итерационные методы поиска минимума.

Цель настоящей работы — рассмотреть основные идеи обучения с учителем и показать, как методы математического анализа возникают в них естественным образом. В качестве основных примеров рассматриваются линейная и логистическая регрессии, градиентные методы оптимизации, решающие деревья и градиентный бустинг.

Для достижения поставленной цели необходимо решить следующие задачи:

1) сформулировать задачу обучения с учителем и ввести основные понятия машинного обучения;

2) рассмотреть линейную регрессию как задачу минимизации среднеквадратичной ошибки;

3) рассмотреть логистическую регрессию и вывести функцию потерь из принципа максимального правдоподобия;

4) описать градиентный спуск и его основные модификации;

5) рассмотреть принцип построения решающих деревьев и объяснить отличие этой задачи от гладкой параметрической оптимизации;

6) рассмотреть градиентный бустинг как последовательное улучшение модели;

7) реализовать основные рассматриваемые методы на Python с использованием NumPy и провести вычислительные эксперименты.

Основой для теоретической части служат лекционные материалы по машинному обучению, учебные материалы Школы анализа данных Яндекса и исходный план проекта [1, 2, 3, 4, 5]. Для раздела, посвящённого современным градиентным оптимизаторам, дополнительно используются стандартные определения, согласующиеся с изложением в [7].

1. Постановка задачи машинного обучения

1.1. Объекты, признаки и ответы

Рассмотрим обучающую выборку

$$D = \{(x_i, y_i)\}_{i=1}^{n},$$

где x_i — описание i-го объекта, а y_i — известный для него ответ.

Будем считать, что каждый объект представлен вектором численных признаков

$$x_i \in \mathbb{R}^d.$$

Координаты этого вектора называются *признаками* объекта. Множество допустимых значений ответа обозначим через $\mathcal{Y}$.

В зависимости от вида $\mathcal{Y}$ различают две основные задачи обучения с учителем.

Регрессия. Если

$$\mathcal{Y} = \mathbb{R},$$

требуется по признакам объекта предсказать вещественное число. Например, по характеристикам квартиры можно предсказывать её стоимость.

Классификация. Если

$$\mathcal{Y} = \{1, \ldots, K\},$$

ответом является номер одного из K классов. Например, банковская операция может относиться к классу обычных или подозрительных.

Таким образом, задача состоит в построении отображения

$$f : \mathbb{R}^d \to \mathcal{Y},$$

которое хорошо восстанавливает зависимость между признаками объектов и ответами.

1.2. Модель как семейство функций

Искать решение среди всех возможных отображений практически невозможно. Поэтому заранее выбирается параметризованное семейство функций

$$\mathcal{F} = \{f_\theta : \theta \in \Theta\},$$

где θ — набор параметров модели.

Важно различать семейство моделей и конкретную обученную модель. Например, множество всех прямых на плоскости является семейством, а одна конкретная прямая соответствует конкретному набору параметров.

Обучение модели означает выбор такого значения параметров $\hat{\theta}$, которое определяется по обучающей выборке:

$$D \longmapsto \hat{\theta} \longmapsto f_{\hat{\theta}}.$$

1.3. Функция потерь

Чтобы формально определить, какие параметры считать хорошими, вводится функция потерь

$$\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_{\geq 0},$$

где $\ell(y, \hat{y})$ измеряет ошибку одного предсказания.

Среднюю ошибку модели на обучающей выборке можно записать в виде

$$L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell\big(y_i, f_\theta(x_i)\big).$$

После фиксации данных L зависит уже не от объектов выборки, а от параметров модели. Если

$$\theta = (\theta_1, \ldots, \theta_m) \in \mathbb{R}^m,$$

то

$$L : \mathbb{R}^m \to \mathbb{R}$$

является функцией нескольких переменных.

В этой постановке обучение параметрической модели принимает вид задачи оптимизации:

$$\boxed{\hat{\theta} \in \arg\min_{\theta \in \Theta} L(\theta)}.$$

Тем самым основные понятия математического анализа возникают непосредственно из постановки задачи обучения: для поиска минимума необходимо исследовать поведение функции нескольких переменных.

1.4. Обучающая и тестовая выборки

Минимизация ошибки на тех же данных, на которых происходило обучение, сама по себе не гарантирует хорошей работы модели на новых объектах. Поэтому исходный набор данных обычно делят как минимум на две части:

- **обучающую выборку** — для подбора параметров модели;
- **тестовую выборку** — для итоговой проверки качества на данных, которые не использовались при обучении.

При необходимости дополнительно выделяется валидационная выборка, на которой выбираются гиперпараметры и сравниваются варианты модели.

2. Линейные модели

2.1. Линейная регрессия

Одним из простейших параметрических семейств является семейство линейных функций. Для удобства свободный коэффициент можно включить в вектор параметров, добавив к вектору признаков единичную координату. Тогда

$$x \in \mathbb{R}^d, \qquad \theta \in \mathbb{R}^d,$$

а предсказание имеет вид

$$\hat{y} = f_\theta(x) = x^T\theta.$$

Для одного численного признака эта запись эквивалентна привычной прямой

$$\hat{y} = wx + b.$$

При этом слово «линейная» относится к зависимости от параметров модели. При добавлении в признаки, например, величины x^2, модель

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2$$

остаётся линейной по параметрам $\theta_0, \theta_1, \theta_2$.

2.2. Среднеквадратичная ошибка

Для регрессии естественно измерять отклонение предсказания от истинного ответа:

$$r_i = y_i - \hat{y}_i.$$

Простое суммирование остатков неудобно, поскольку положительные и отрицательные ошибки могут взаимно компенсироваться. Один из стандартных вариантов — использовать квадратичную функцию потерь

$$\ell(y, \hat{y}) = (y - \hat{y})^2.$$

Тогда среднеквадратичная ошибка имеет вид

$$L(\theta) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - x_i^T\theta\right)^2.$$

Если составить матрицу признаков

$$X = \begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix} \in \mathbb{R}^{n\times d}$$

и вектор ответов

$$Y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix},$$

то функцию потерь можно компактно записать:

$$L(\theta) = \frac{1}{n}\|Y - X\theta\|_2^2.$$

В простейшем случае $\theta = (w, b)$ получаем функцию двух переменных

$$L = L(w, b).$$

Её можно визуализировать как поверхность, а множества

$$L(w, b) = C$$

— как линии уровня. Минимум поверхности соответствует параметрам наилучшей прямой в смысле MSE.

PLACEHOLDER

Поверхность $L(w, b)$ и линии уровня для линейной регрессии.

2.3. Градиент среднеквадратичной ошибки

Для дальнейшего применения градиентных методов достаточно получить градиент MSE по параметрам модели. Для

$$L(\theta) = \frac{1}{n}\|Y - X\theta\|_2^2$$

имеем

$$\boxed{\nabla_\theta L(\theta) = \frac{2}{n} X^T (X\theta - Y)}.$$

Эта формула показывает, как изменение каждого параметра влияет на значение функции потерь. В частном случае линейной регрессии минимум MSE можно найти и аналитически, однако для общей логики работы далее важнее численный подход: имея значение градиента, можно последовательно изменять параметры в направлении уменьшения функции потерь.

2.4. Логистическая регрессия

Рассмотрим бинарную классификацию:

$$y_i \in \{0, 1\}.$$

Линейная комбинация признаков

$$s_\theta(x) = x^T\theta$$

может принимать любые вещественные значения, поэтому напрямую интерпретировать её как вероятность нельзя.

В логистической регрессии вводится предположение о линейности логарифма шансов:

$$\log \frac{p_\theta(x)}{1 - p_\theta(x)} = x^T\theta.$$

Отсюда

$$\frac{p}{1-p} = e^{x^T\theta},$$

поэтому

$$p_\theta(x) = \frac{1}{1 + e^{-x^T\theta}} = \sigma(x^T\theta),$$

где

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

— сигмоидальная функция.

Таким образом,

$$p_\theta(x) = P_\theta(Y = 1 \mid x).$$

При стандартном пороге $1/2$ класс определяется правилом

$$\hat{y}(x) = \begin{cases} 1, & p_\theta(x) \geq \frac{1}{2}, \\ 0, & p_\theta(x) < \frac{1}{2}. \end{cases}$$

PLACEHOLDER

График сигмоиды и интерпретация вероятности класса.

2.5. Максимальное правдоподобие и cross-entropy

Пусть

$$p_i = p_\theta(x_i).$$

При условной независимости наблюдений вероятность получить конкретные ответы $y_1, \ldots, y_n$ при заданных параметрах θ записывается как

$$\mathcal{L}(\theta) = \prod_{i=1}^{n} p_i^{y_i}(1 - p_i)^{1-y_i}.$$

Метод максимального правдоподобия выбирает параметры

$$\hat{\theta} \in \arg\max_\theta \mathcal{L}(\theta).$$

Поскольку логарифм является монотонно возрастающей функцией, максимум правдоподобия совпадает с максимумом логарифма правдоподобия:

$$\log \mathcal{L}(\theta) = \sum_{i=1}^{n} \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right].$$

Переходя от максимизации к минимизации отрицательного лог-правдоподобия, получаем функцию потерь

$$\boxed{L(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log p_i + (1-y_i)\log(1-p_i)\right]}.$$

Это бинарная cross-entropy.

Несмотря на изменение модели и функции потерь, общая математическая постановка остаётся прежней:

$$\hat{\theta} \in \arg\min_{\theta} L(\theta).$$

Именно поэтому один и тот же класс численных методов оптимизации может применяться к различным моделям.

3. Градиентные методы оптимизации

3.1. Градиентный спуск

Пусть

$$L : \mathbb{R}^d \to \mathbb{R}$$

— дифференцируемая функция потерь. Её градиент в точке θ определяется как

$$\nabla L(\theta) = \begin{pmatrix} \dfrac{\partial L}{\partial \theta_1} \\ \vdots \\ \dfrac{\partial L}{\partial \theta_d} \end{pmatrix}.$$

Градиент задаёт направление наиболее быстрого локального роста функции. Поэтому для уменьшения значения L естественно двигаться в противоположном направлении.

Итерация градиентного спуска:

$$\boxed{\theta_{t+1} = \theta_t - \eta_t \nabla L(\theta_t)},$$

где $\eta_t > 0$ называется *скоростью обучения* или *learning rate*.

На двумерной функции потерь траекторию алгоритма удобно изображать на линиях уровня.

PLACEHOLDER

Траектория Gradient Descent на линиях уровня функции потерь.

3.2. Влияние размера шага

Значение η существенно влияет на поведение метода.

Рассмотрим одномерный пример

$$f(u) = u^2.$$

Тогда

$$f'(u) = 2u$$

и итерация градиентного спуска имеет вид

$$u_{t+1} = u_t - 2\eta u_t = (1 - 2\eta)u_t.$$

Из этой формулы непосредственно видно, что при различных η поведение последовательности может существенно отличаться: она может медленно сходиться, колебаться около минимума или расходиться.

В практической части рассматриваются три характерных режима работы градиентного спуска:

- слишком маленький шаг;
- шаг, обеспечивающий устойчивую быструю сходимость;
- слишком большой шаг.

PLACEHOLDER

Сравнение траекторий GD при различных значениях learning rate.

3.3. Momentum

На вытянутых поверхностях уровня обычный градиентный спуск может совершать сильные колебания поперёк узкой долины. Метод Momentum добавляет к текущему направлению информацию о предыдущих шагах:

$$v_t = \beta v_{t-1} + g_t, \qquad g_t = \nabla L(\theta_t),$$

$$\theta_{t+1} = \theta_t - \eta v_t.$$

Параметр $\beta \in [0, 1)$ определяет степень сохранения накопленного направления. За счёт этого последовательные градиенты усиливают движение вдоль устойчивого направления и частично компенсируют поперечные колебания.

3.4. Nesterov Momentum

Метод Нестерова использует идею “заглядывания вперёд”: градиент оценивается не только в текущей точке, но с учётом предполагаемого движения по накопленному импульсу.

Одна из стандартных форм записи:

$$g_t = \nabla L\left(\theta_t - \eta\beta v_{t-1}\right),$$

$$v_t = \beta v_{t-1} + g_t,$$

$$\theta_{t+1} = \theta_t - \eta v_t.$$

В результате алгоритм раньше реагирует на изменение направления поверхности функции потерь.

3.5. AdaGrad

В многомерных задачах функция может изменяться с различной скоростью по разным координатам. AdaGrad использует отдельный эффективный размер шага для каждой координаты.

Пусть

$$g_t = \nabla L(\theta_t).$$

Накопим суммы квадратов градиентов:

$$G_t = G_{t-1} + g_t \odot g_t,$$

где $\odot$ обозначает покоординатное умножение.

Обновление параметров:

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{G_t} + \varepsilon},$$

где деление и квадратный корень выполняются покоординатно.

Недостаток метода состоит в том, что G_t постоянно возрастает, поэтому эффективный шаг со временем может стать слишком малым.

3.6. RMSProp

RMSProp заменяет полное накопление квадратов градиентов экспоненциальным скользящим средним:

$$s_t = \rho s_{t-1} + (1-\rho) g_t \odot g_t.$$

Параметры обновляются по правилу

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_t} + \varepsilon}.$$

В отличие от AdaGrad, старые значения постепенно теряют влияние, поэтому шаг не обязан монотонно стремиться к нулю.

3.7. Adam

Adam объединяет идеи Momentum и RMSProp. Вводятся экспоненциальные средние первого и второго моментов градиента:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t \odot g_t.$$

Так как в начале $m_0 = v_0 = 0$, применяются поправки на смещение:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

Итоговое обновление:

$$\boxed{\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}}.$$

Таким образом, Adam одновременно использует накопленное направление движения и адаптивный масштаб шага по отдельным координатам.

3.8. Сравнение методов

Для корректного визуального сравнения рассматриваемые оптимизаторы запускаются:

- на одной функции;
- из одной стартовой точки;
- при заранее зафиксированных гиперпараметрах.

Сравниваются следующие методы:

GD, Momentum, Nesterov, AdaGrad, RMSProp, Adam.

Основными объектами сравнения являются траектории на линиях уровня и изменение функции потерь по итерациям.

PLACEHOLDER

Траектории шести оптимизаторов из одной стартовой точки.

4. Решающие деревья и градиентный бустинг

4.1. Решающие деревья

Решающее дерево строит предсказание последовательностью простых условий. Для вещественного признака типичный предикат имеет вид

$$x_j \le t,$$

где j — номер признака, а t — порог.

Пусть в некоторую вершину дерева пришло множество объектов S. Кандидат на разбиение (j, t) делит его на две части:

$$S_L(j,t) = \{(x_i, y_i) \in S : \ x_{ij} \le t\},$$

$$S_R(j,t) = \{(x_i, y_i) \in S : \ x_{ij} > t\}.$$

Задача алгоритма в этой вершине — выбрать такое разбиение, после которого объекты в дочерних вершинах становятся как можно более однородными по ответам.

4.2. Критерий разбиения

Обозначим через $H(S)$ меру неоднородности множества объектов в вершине. Для разбиения (j, t) можно определить качество как уменьшение неоднородности:

$$Q(S,j,t) = H(S) - \frac{|S_L|}{|S|}H(S_L) - \frac{|S_R|}{|S|}H(S_R).$$

На каждом шаге дерево выбирает

$$(j^*, t^*) \in \arg\max_{j,t} Q(S,j,t).$$

Таким образом, рассматриваются возможные признаки и пороги, после чего жадно выбирается локально лучшее разбиение.

Регрессия. В задаче регрессии в качестве неоднородности естественно использовать среднеквадратичное отклонение ответов от их среднего:

$$H_{\text{MSE}}(S) = \frac{1}{|S|}\sum_{(x_i,y_i)\in S}(y_i - \bar{y}_S)^2,$$

где

$$\bar{y}_S = \frac{1}{|S|}\sum_{(x_i,y_i)\in S} y_i.$$

Эквивалентно можно выбирать признак j и порог t, минимизирующие сумму квадратов ошибок в двух дочерних вершинах:

$$\min_{j,t}\left[\sum_{(x_i,y_i)\in S_L}(y_i - \bar{y}_L)^2 + \sum_{(x_i,y_i)\in S_R}(y_i - \bar{y}_R)^2\right].$$

Предсказанием в листе регрессионного дерева обычно служит среднее значение ответов объектов, попавших в этот лист.

Классификация. Для классификации часто используют критерий Джини

$$G(S) = 1 - \sum_{k=1}^{K} p_k^2$$

или энтропию

$$H(S) = -\sum_{k=1}^{K} p_k \log p_k,$$

где p_k — доля объектов класса k в вершине.

Чем меньше неоднородность дочерних вершин, тем удачнее разбиение. В листе классификационного дерева можно возвращать наиболее частый класс или оценки вероятностей классов по их долям среди объектов листа.

4.3. Жадное построение дерева

Полный перебор всех возможных структур дерева практически невозможен, поэтому дерево строится жадно:

1. в текущую вершину приходит множество объектов S;
2. перебираются допустимые признаки j и пороги t;
3. для каждого кандидата вычисляется значение критерия разбиения;
4. выбирается лучшее разбиение (j^*, t^*);
5. процедура рекурсивно повторяется для левой и правой дочерних вершин.

Рост дерева останавливают, например, при достижении максимальной глубины, слишком малом числе объектов в вершине или недостаточном уменьшении неоднородности. Эти ограничения позволяют контролировать сложность дерева и уменьшать переобучение.

4.4. Почему обычный Gradient Descent здесь неприменим

В линейных моделях параметры принадлежат непрерывному пространству $\mathbb{R}^d$, а функция потерь рассматривается как функция этих параметров.

При построении дерева необходимо выбирать номер признака, порог и саму дискретную структуру дерева. Поэтому нельзя непосредственно вычислить производную функции потерь, например, по номеру выбранного признака. Вместо движения по антиградиенту используется жадный перебор допустимых локальных разбиений.

4.5. Случайность и разнообразие деревьев

Для *одного* обычного решающего дерева случайный выбор признаков не является обязательной частью алгоритма: при поиске split'а можно рассматривать все доступные признаки.

Случайный выбор объектов и признаков становится особенно важен в ансамблях независимых деревьев. Например, в Random Forest каждое дерево строится на

собственной bootstrap-выборке объектов, а при поиске разбиения в вершине рассматривается случайное подмножество признаков. Благодаря этому отдельные деревья получаются менее похожими друг на друга, а усреднение их предсказаний уменьшает разброс ансамбля.

В Gradient Boosting деревья различаются по другой причине: каждое следующее дерево обучается уже на новой задаче, зависящей от ошибок текущего ансамбля. Дополнительный случайный subsampling объектов или признаков также может применяться, но не является обязательным для самой идеи градиентного бустинга.

4.6. Идея ансамбля

Вместо одной модели можно построить композицию нескольких базовых алгоритмов:

$$F(x) = \sum_{m=1}^{M} \alpha_m h_m(x),$$

где h_m — отдельные базовые модели.

В Random Forest деревья строятся в значительной степени независимо, после чего их ответы усредняются или объединяются голосованием. Градиентный бустинг использует другую схему: деревья строятся последовательно, и каждый следующий алгоритм старается исправить ошибку уже построенной композиции.

4.7. Gradient Boosting

Пусть после $m-1$ шагов имеется модель

$$F_{m-1}(x).$$

На следующем шаге к ней добавляется новое дерево:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x),$$

где η — размер шага бустинга.

Для квадратичной функции потерь

$$\ell(y, F) = (y - F)^2$$

отрицательный градиент по текущему предсказанию пропорционален обычному остатку

$$y_i - F_{m-1}(x_i).$$

Поэтому в этом случае очередное дерево можно обучать приближать residuals текущего ансамбля.

В общем случае вводятся псевдоостатки

$$r_i^{(m)} = - \left. \frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_{m-1}}.$$

Новое дерево h_m обучается приближать эти значения, после чего добавляется к ансамблю. Тем самым идея антиградиента переносится с пространства обычных параметров на последовательное построение самой функции F.

PLACEHOLDER

Пошаговое добавление деревьев и изменение residuals.

5. Дополнительные методы

Основная часть работы посвящена обучению с учителем и градиентным методам. Дополнительно рассматриваются несколько подходов, использующих другие математические идеи [4].

5.1. Principal Component Analysis

PCA применяется для уменьшения размерности данных. Пусть строки матрицы X содержат центрированные объекты. Рассмотрим ковариационную матрицу

$$C = \frac{1}{N} X^T X.$$

Первая главная компонента определяется направлением единичной длины, вдоль которого дисперсия проекции максимальна:

$$\max_{\|w\|=1} w^T C w.$$

Решением является собственный вектор матрицы C, соответствующий наибольшему собственному значению. Геометрически PCA выбирает направления, вдоль которых данные обладают наибольшим разбросом, и позволяет отбросить направления с относительно небольшой вариативностью.

5.2. Метод Монте-Карло

Методы Монте-Карло используют случайную выборку для приближённого вычисления математических величин. Например,

$$I = \int_a^b f(x)\,dx.$$

Если

$$X \sim U(a, b),$$

то

$$I = (b - a)\mathbb{E}[f(X)].$$

Поэтому интеграл можно приближённо оценить по случайным точкам:

$$I \approx \frac{b-a}{N} \sum_{i=1}^{N} f(x_i).$$

Эта идея важна не только для численного интегрирования: случайные оценки математических ожиданий постоянно возникают в статистике, оптимизации и машинном обучении.

5.3. Муравьиный алгоритм

Ant Colony Optimization является вероятностным метаэвристическим методом оптимизации. Он не относится непосредственно к рассматриваемым выше алгоритмам обучения с учителем, но служит примером другого подхода к поиску сложного решения.

Вероятность перехода из вершины i в вершину j можно задавать в виде

$$P_{ij} = \frac{\tau_{ij}^{\alpha} \eta_{ij}^{\beta}}{\sum\limits_{k \in A_i} \tau_{ik}^{\alpha} \eta_{ik}^{\beta}},$$

где τ_{ij} описывает накопленный феромон, η_{ij} — локальную привлекательность перехода, а A_i — множество допустимых переходов.

После итерации феромон обновляется:

$$\tau_{ij} \leftarrow (1-\rho)\tau_{ij} + \Delta\tau_{ij}.$$

Работа метода демонстрируется на небольшой задаче коммивояжёра, где отслеживаются изменение феромонов, вероятностей переходов и найденного маршрута.

6. Практическая часть

6.1. Используемые средства

Практическая часть реализуется на языке Python. Основными используемыми библиотеками являются:

- `NumPy` — для вычислений с массивами и реализации алгоритмов;
- `Matplotlib` — для построения графиков и визуализации траекторий;
- `Pandas` — при необходимости для удобного представления табличных данных.

Основные алгоритмы обучения реализуются самостоятельно. Готовые реализации из `scikit-learn` используются только для дополнительной проверки результатов и не заменяют собственную реализацию исследуемых алгоритмов.

6.2. Эксперимент 1: поверхность loss линейной регрессии

Для простого набора точек рассматривается модель

$$\hat{y} = wx + b$$

и функция

$$L(w, b) = \frac{1}{n} \sum_{i=1}^{n} \big(y_i - wx_i - b\big)^2.$$

В ходе эксперимента выполняются следующие действия:

1. построить исходные данные;
2. построить поверхность $L(w, b)$;
3. показать линии уровня;
4. отметить положение минимума;
5. визуализировать изменение прямой при движении параметров к минимуму.

PLACEHOLDER

Итоговая поверхность MSE и положение найденного минимума.

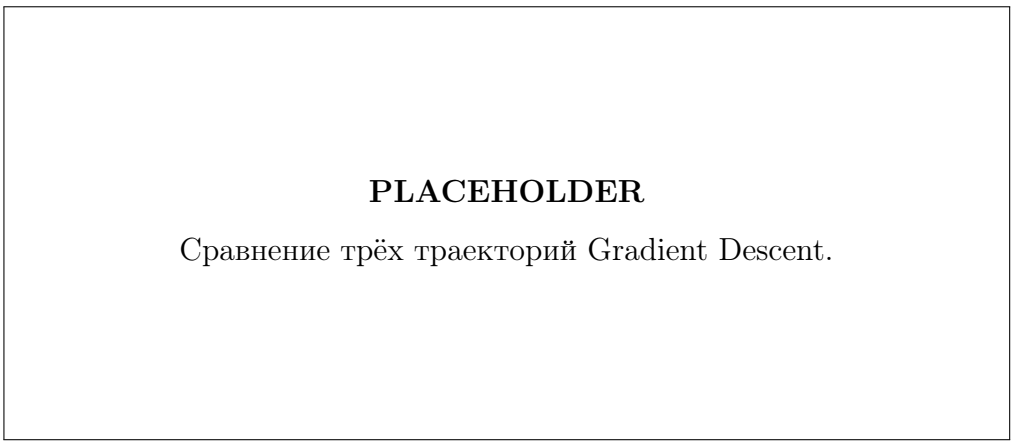
Результат эксперимента. [PLACEHOLDER: результаты эксперимента и численные значения]

6.3. Эксперимент 2: влияние learning rate

Из одной стартовой точки запускается Gradient Descent с несколькими значениями η.

Рассматриваются три типичных режима:

- медленная сходимость при малом шаге;
- устойчивая сходимость при подходящем шаге;
- колебания или расходимость при слишком большом шаге.



Результат эксперимента. [PLACEHOLDER: значения η и анализ полученных траекторий]

6.4. Эксперимент 3: сравнение оптимизаторов

На одной двумерной функции из одной начальной точки запускаются:

$$\text{GD}, \text{Momentum}, \text{Nesterov}, \text{AdaGrad}, \text{RMSProp}, \text{Adam}.$$

Для каждого метода фиксируются:

- число итераций;
- траектория;
- значения функции потерь;
- расстояние до известного минимума, если оно может быть вычислено.

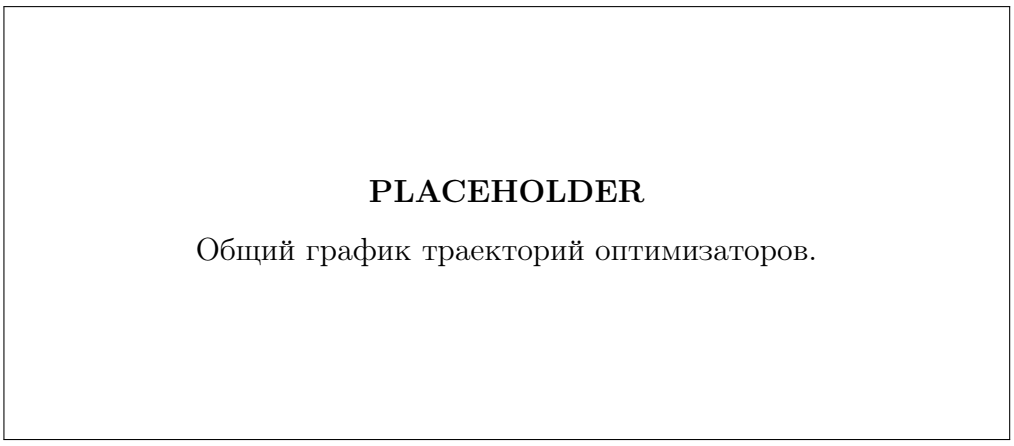


Таблица 1: Сравнение оптимизаторов

Метод	Итерации	Финальный loss	Комментарий
GD	PLACEHOLDER	PLACEHOLDER	PLACEHOLDER
Momentum	PLACEHOLDER	PLACEHOLDER	PLACEHOLDER
Nesterov	PLACEHOLDER	PLACEHOLDER	PLACEHOLDER
AdaGrad	PLACEHOLDER	PLACEHOLDER	PLACEHOLDER
RMSProp	PLACEHOLDER	PLACEHOLDER	PLACEHOLDER
Adam	PLACEHOLDER	PLACEHOLDER	PLACEHOLDER

6.5. Эксперимент 4: построение небольшого дерева

На небольшом наборе данных воспроизводятся несколько первых шагов построения дерева:

1. перебрать возможные признаки и пороги;
2. вычислить значение критерия для каждого кандидата;
3. выбрать лучшее разбиение;
4. повторить процедуру в дочерних вершинах.

PLACEHOLDER

Исходные данные и выбранные границы разбиения дерева.

Результат эксперимента. [PLACEHOLDER: выбранные разбиения и значения критерия]

6.6. Эксперимент 5: пошаговый Gradient Boosting

Для демонстрации работы градиентного бустинга рассматривается последовательность

$$F_0, F_1, \ldots, F_M.$$

На каждом шаге сохраняются:

- текущие предсказания;
- residuals или pseudo-residuals;

- новое дерево;
- значение общей функции потерь.

Итоговая визуализация показывает изменение функции потерь при увеличении числа деревьев.

PLACEHOLDER

График loss в зависимости от числа добавленных деревьев.

Результат эксперимента. [PLACEHOLDER: результаты эксперимента]

6.7. Дополнительные демонстрации

В качестве дополнительных демонстраций рассматриваются следующие визуализации:

- PCA: двумерное облако точек, главные направления и проекция на первую компоненту;
- Monte Carlo: приближение интеграла при возрастающем числе случайных точек;
- Ant Colony Optimization: изменение феромонов и маршрута в небольшой задаче коммивояжёра.

Дополнительные демонстрации иллюстрируют методы, основанные на других математических идеях, и дополняют основную практическую часть по линейным моделям, оптимизаторам, деревьям и градиентному бустингу.

7. Распределение работы между участниками

Работа над проектом распределена между участниками следующим образом:

- **Зайцев Н.** — составление отчёта и подготовка презентации;
- **Вейгандт В.** — написание программного кода и реализация рассматриваемых алгоритмов;
- **Козлов В.** — создание анимаций и визуализаций, сопровождающих основные алгоритмы;

- **Горобец О.** — подготовка и вывод математических доказательств и выкладок;
- **Шагеев Э.** — танец продматника.

Заключение

В работе была сформулирована базовая задача обучения с учителем и рассмотрены несколько ключевых методов классического машинного обучения.

На примере линейной регрессии было показано, что обучение параметрической модели может быть представлено как поиск минимума функции нескольких переменных. Среднеквадратичная ошибка задаёт функцию от параметров модели, а её градиент определяет направление локального изменения ошибки и естественно приводит к численным методам оптимизации.

Для задачи бинарной классификации была рассмотрена логистическая регрессия. Функция cross-entropy была получена из принципа максимального правдоподобия, после чего задача обучения вновь свелась к минимизации функции потерь.

Градиентный спуск был рассмотрен как общий численный метод минимизации. Его модификации — Momentum, Nesterov, AdaGrad, RMSProp и Adam — изменяют способ использования информации о градиентах и позволяют улучшать движение по сложным поверхностям функции потерь.

Решающие деревья представляют другой тип моделей: их структура выбирается в дискретном пространстве, поэтому обычный градиентный спуск непосредственно к построению дерева не применяется. Был подробно рассмотрен жадный выбор разбиений и роль критериев неоднородности. Также было отмечено, что случайный выбор объектов и признаков характерен прежде всего для Random Forest, тогда как в Gradient Boosting разнообразие деревьев возникает за счёт последовательного исправления ошибок текущего ансамбля.

Дополнительно были кратко рассмотрены PCA, метод Монте-Карло и муравьиный алгоритм как примеры применения линейной алгебры, вероятностных идей и метаэвристической оптимизации.

Таким образом, математический анализ не является отдельным внешним дополнением к рассматриваемым алгоритмам. Понятия функции нескольких переменных, частных производных, градиента, экстремума и сходимости естественно возникают непосредственно из задачи обучения моделей.

Вычислительная часть дополняет теоретическое рассмотрение экспериментальным сравнением поведения изученных методов.

Список литературы

[1] Броиловский А. *Лекция 1. Обзор задач машинного обучения.* По мотивам лекции Н. Волкова, ШАД СПб, 2026.

[2] Овчаров С. *Лекция 2. Линейная регрессия и основы оптимизации.* По мотивам лекции Н. Волкова, ШАД СПб, 2026.

[3] Овчаров С. *Лекция 3. Логистическая регрессия и классификация.* По мотивам лекции Н. Волкова, ШАД СПб, 2026.

[4] *Математический анализ в методах машинного обучения*. Материалы проекта, 2026.

[5] Школа анализа данных Яндекса. *Учебник по машинному обучению*. https://contest.yandex.ru/tracks/ml/.

[6] Синицин Ф., Соколов Е. *Линейные модели*. Учебник по машинному обучению ШАД Яндекса. https://contest.yandex.ru/tracks/ml/supervised-learning/linear-models.

[7] Zhang A., Lipton Z. C., Li M., Smola A. J. *Dive into Deep Learning. Chapter 12: Optimization Algorithms*. https://d2l.ai/chapter_optimization/index.html.

[8] Школа анализа данных Яндекса. *Решающие деревья*. https://contest.yandex.ru/tracks/ml/supervised-learning/decision-trees.

[9] Школа анализа данных Яндекса. *Градиентный бустинг*. https://contest.yandex.ru/tracks/ml/supervised-learning/gradient-boosting.

[10] Scikit-learn Developers. *Ensembles: Gradient boosting, random forests, bagging, voting, stacking*. https://scikit-learn.org/stable/modules/ensemble.html.

A. Программная реализация

Исходный код проекта доступен в репозитории:

[PLACEHOLDER: ссылка на репозиторий]